

Rapport de projet

Optical Character Recognition

Destructeur 2 Mots



Kristen Couty - Romain Dischamp - Liam Chevassus
Timothée Gallier

D2

Octobre 2025

Tables des matières

1	Présentation du projet	2
1.1	Nature du projet	2
1.2	Membres	3
1.2.1	Kristen Couty	3
1.2.2	Romain Dischamp	3
1.2.3	Liam Chevassus	3
1.2.4	Timothée Gallier	3
2	Bilan des réalisations	5
2.1	Traitement de l'image	5
2.1.1	Bilan des réalisations	5
2.1.2	Défis rencontrés	8
2.2	Détection des lignes	9
2.2.1	Bilan des réalisations	9
2.2.2	Défis rencontrés	11
2.3	Réseau de neurones	12
2.3.1	Bilan des réalisations	12
2.3.2	Défis rencontrés	13
2.4	Algorithme de résolution	14
2.4.1	Bilan des réalisations	14
2.4.2	Défis rencontrés	14
2.5	Interface graphique	15
2.5.1	Bilan des réalisations	15
2.5.2	Défis rencontrés	16
3	Gestion de projet	17
3.1	Répartition et avancement du travail	17
4	Conclusion	18
5	Annexes	19
5.0.1	Termes techniques et définitions	19
5.0.2	Bibliographie	20

1 Présentation du projet

1.1 Nature du projet

Ce projet vise à développer un logiciel innovant de reconnaissance optique de caractères (*OCR*) capable de résoudre automatiquement des grilles de mots cachés. Les grilles de mots cachés, très populaires comme jeu de réflexion, consistent en un tableau de lettres dans lequel il faut identifier des mots dissimulés horizontalement, verticalement ou en diagonale. L'objectif de notre application est de prendre en entrée une image représentant une telle grille et d'en produire une version entièrement résolue.

Dans sa version finale, le logiciel offrira une interface graphique intuitive permettant à l'utilisateur de charger des images dans des formats standards, de visualiser la grille, et d'appliquer des corrections sur l'image si nécessaire pour améliorer la qualité de reconnaissance. Une fois traitée, la grille pourra être affichée sous sa forme complète et résolue, avec la possibilité de sauvegarder le résultat pour une utilisation ultérieure.

Un aspect essentiel de ce projet réside dans l'intégration d'un module d'apprentissage automatique. Ce module permettra de créer et d'entraîner un réseau de neurones capable de reconnaître les caractères présents dans les grilles, avec la possibilité de sauvegarder et de recharger les modèles d'apprentissage afin d'améliorer les performances du logiciel au fil du temps.

Ainsi, ce projet combine à la fois des compétences en traitement d'images, en intelligence artificielle et en conception d'interfaces utilisateur, offrant une solution complète et interactive pour résoudre automatiquement des grilles de mots cachés à partir d'images.

1.2 Membres

1.2.1 Kristen Couty

Dans ce projet, je suis en charge de la conception et du développement du réseau de neurones qui est au cœur de la partie d'apprentissage automatique de l'application. Ma mission consiste à créer un modèle capable de reconnaître et d'identifier avec précision les caractères présents dans les grilles de mots cachés, puis de l'entraîner sur des ensembles de données adaptés afin d'améliorer ses performances au fil du temps.

En parallèle, je m'occupe également de la coordination du projet, veillant à la cohérence entre les différentes parties du logiciel et au respect des objectifs fixés, tout en facilitant la collaboration entre les membres de l'équipe. Mon rôle combine donc à la fois un aspect technique, lié à l'intelligence artificielle, et un aspect organisationnel, garantissant la progression fluide et structurée du projet.

1.2.2 Romain Dischamp

Mes missions principales dans ce projet sont les suivantes : Tout d'abord, je dois concevoir un algorithme capable de prendre en entrée un tableau de caractères représentant la grille de mots cachés, ainsi qu'un mot à rechercher. L'objectif de cet algorithme est de déterminer si le mot est présent dans la grille et de renvoyer les coordonnées de début et de fin du mot trouvé. Cet algorithme constitue la dernière brique de la grande pipeline du projet, et joue donc un rôle essentiel dans le bon fonctionnement de l'ensemble. Je suis aussi chargé du développement d'un programme de détection et d'extraction de caractères présents dans les images du dataset. Le but étant d'extraire toutes les lettres, pour que le futur OCR qui sera réalisé par Kristen analyse chaque image de lettre, et renvoie le caractère associé.

1.2.3 Liam Chevassus

Dans ce projet, je suis responsable du traitement de l'image et de l'interface graphique. Ainsi, je suis en charge du chargement et du traitement de l'image, ce qui regroupe diverses opérations effectuées sur une image : chargement de l'image dans l'interface graphique, conversion en noir et blanc et rotation de l'image. De plus, mon travail est également d'améliorer la qualité de l'image, en supprimant les grains, par exemple.

Par conséquent, mon rôle est de faire le prétraitement de l'image, afin que l'image puisse être envoyée au réseau de neurones pour que la grille soit résolue. L'image doit donc être de la plus haute qualité possible. Enfin, je contribue également au développement de l'interface graphique, qui permet à l'utilisateur de voir le résultat final avec la grille résolue, mais aussi d'agir sur certains paramètres. Je m'attache donc à ce que l'expérience utilisateur soit la plus agréable possible, en produisant une solution combinant ergonomie de l'interface graphique et efficacité du processus de résolution de la grille.

1.2.4 Timothée Gallier

Dans ce projet, je suis en charge d'une partie du traitement de l'image, sur tout ce qui est de la découpe de l'image. Cette découpe est très importante, car elle permettra

de donner les bonnes images et les bons caractères au réseau de neurones afin que celui-ci s'entraîne correctement et ne fasse pas d'erreurs liées à une mauvaise qualité de l'image. Je suis également en charge de la résolution et de la reconstruction des différentes grilles. Mon rôle est donc principalement technique, sur la gestion de la librairie *GTK* et *GDK* afin de donner le meilleur au réseau de neurones pour que celui-ci soit efficace lors de la réalisation de sa tâche.

2 Bilan des réalisations

2.1 Traitement de l'image

2.1.1 Bilan des réalisations

Traitements

Avant tout traitement de l'image, il faut d'abord charger cette image et y effectuer un prétraitement. Ce prétraitement inclut deux opérations principales :

- Chargement de l'image dans l'interface graphique.
- Conversion de l'image :
 - Conversion de l'image en niveaux de gris.
 - Puis conversion de l'image en noir et blanc.

Le chargement de l'image se fait en créant une nouvelle application *GTK*, dans laquelle on rajoute l'image à charger dans une box. On ajoute cette box contenant l'image dans la fenêtre de l'application, ce qui affiche l'image. On conserve d'ailleurs le *GdkPixbuf* associé à cette image.

Quant à la conversion en niveaux de gris, on parcourt chaque pixel du *GdkPixbuf*, et on applique la formule suivante :

$$Gris = 0,299 \times R + 0,587 \times G + 0,114 \times B$$

Une fois l'image convertie en niveaux de gris, on peut maintenant la convertir en noir et blanc, en appliquant un processus appelé binarisation de l'image. Celui-ci utilise un seuil, généralement 128. Chaque pixel en dessous de ce seuil (en niveaux de gris) devient noir (R, G et B sont à 0), sinon il devient blanc (R, G et B sont à 255).

	P	X	U	T	S	I	N	I	U	P	R	V	G	B	M	D	D
	E	H	A	A	S	P	O	J	P	E	T	B	E	Q	Z	L	C
	A	U	N	T	E	G	Q	T	L	H	R	Z	F	A	T	O	P
	S	H	X	F	N	G	U	A	X	E	A	A	Y	P	O	M	H
IMAGINE	Y	O	Y	Y	L	D	X	L	A	K	Y	U	Z	L	B	S	K
RELAX	J	X	M	U	U	G	Q	T	R	I	M	A	G	I	N	E	B
COOL	H	F	N	W	F	X	H	D	P	B	B	B	T	N	V	S	K
RESTING	H	I	I	H	D	E	S	Q	F	U	M	Y	E	R	N	S	X
BREATHE	R	P	B	Z	N	H	S	D	S	L	H	O	N	B	S	S	S
EASY	E	H	X	A	I	Z	I	H	A	H	O	E	S	Q	F	E	F
TENSION	C	W	Z	I	M	V	D	C	J	V	S	S	I	M	G	R	W
STRESS	L	A	I	I	R	Z	Q	Q	H	X	D	Z	O	Z	Q	T	R
CALM	W	C	A	X	E	Z	R	G	H	A	I	Z	N	E	C	S	E
	B	R	H	F	O	T	G	N	I	T	S	E	R	E	O	V	Z
	M	W	V	W	Q	D	U	I	H	W	Q	T	S	B	I	M	L
	T	D	T	O	N	Z	C	X	X	R	G	E	L	K	H	F	Q
	Q	N	E	K	S	V	M	O	T	F	A	L	A	A	E	W	B

Figure 1: Une image en niveau de gris

	P	X	U	T	S	I	N	I	U	P	R	V	G	B	M	D	D
	E	H	A	A	S	P	O	J	P	E	T	B	E	Q	Z	L	C
	A	U	N	T	E	G	Q	T	L	H	R	Z	F	A	T	O	P
IMAGINE	S	H	X	F	N	G	U	A	X	E	A	A	Y	P	O	M	H
RELAX	Y	O	Y	Y	L	D	X	L	A	K	Y	U	Z	L	B	S	K
COOL	J	X	M	U	U	G	Q	T	R	I	M	A	G	I	N	E	B
RESTING	H	F	N	W	F	X	H	D	P	B	B	B	T	N	V	S	K
BREATHE	H	I	I	H	D	E	S	Q	F	U	M	Y	E	R	N	S	X
EASY	R	P	B	Z	N	H	S	D	S	L	H	O	N	B	S	S	S
TENSION	E	H	X	A	I	Z	I	H	A	H	O	E	S	Q	F	E	F
STRESS	C	W	Z	I	M	V	D	C	J	V	S	S	I	M	G	R	W
CALM	L	A	I	I	R	Z	Q	Q	H	X	D	Z	O	Z	Q	T	R
	W	C	A	X	E	Z	R	G	H	A	I	Z	N	E	C	S	E
	B	R	H	F	O	T	G	N	I	T	S	E	R	E	O	V	Z
	M	W	V	W	Q	D	U	I	H	W	Q	T	S	B	I	M	L
	T	D	T	O	N	Z	C	X	X	R	G	E	L	K	H	F	Q
	Q	N	E	K	S	V	M	O	T	F	A	L	A	A	E	W	B

Figure 2: L'image convertie en noir et blanc

Rotation

De la même manière que pour les conversions expliquées ci-dessus, la rotation de l'image consiste à faire tourner l'image d'un certain angle et à effectuer une rotation sur chaque pixel de l'image d'origine de cet angle, pour donner une image d'arrivée qui a subi cette rotation.

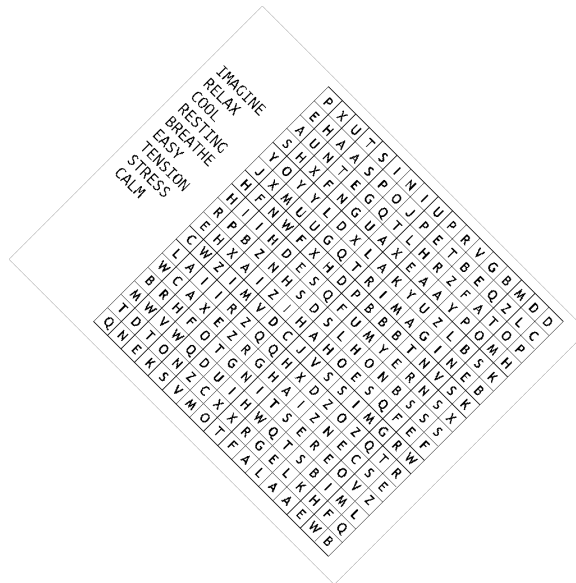


Figure 3: L'image ayant subi une rotation d'angle 45° vers la droite

Découpage des images

Afin que l'OCR final puisse reconnaître les caractères et les résoudre, il est essentiel de lui donner des images bien formatées, ne contenant qu'un caractère à la fois, afin de permettre au réseau de neurones d'apprendre et d'analyser. Il a donc fallu faire des fonctions prenant en charge une image et l'adaptant à différents besoins. La première fonction implémentée a été une découpe de l'image en deux parties, de manière verticale ou horizontale, qui sera utile afin de séparer la partie liste des mots à trouver et la partie contenant la grille de lettres. Les images sont gérées par l'interface *GTK3* qui utilise elle-même une sous-partie appelée *GDK* avec des *GdkPixBuf*, là où sont stockées les images que l'on peut ensuite manipuler et afficher de la manière que nous souhaitons. La 2ème fonction réalisée sera utilisée pour découper des images carrées en parts égales carrées, elles aussi, avec le nombre de découpes souhaitées. La 3ème et dernière fonction pour le moment sert quant à elle à découper un rectangle aux coordonnées souhaitées de (x1, y1) à (x2, y2). Cette fonction sera utilisée notamment pour découper les lettres et extraire les caractères un à un, notamment lors de la détection de ligne.



Figure 4: Exemple utilisation `./build/image crop level_1_image_1.png 100 100 300 300`

2.1.2 Défis rencontrés

Traitements

Pour les traitements, nous avons tout d'abord eu du mal à charger l'image dans l'interface graphique, car il faut suivre un ordre précis dans l'initialisation des pointeurs pour la fenêtre et la boîte contenant l'image. De plus, pour sauvegarder l'image en fichier *PNG*, nous utilisons la fonction `save_pixbuf_as_png(GdkPixbuf *pixbuf, char *filename)` qui permet de transformer un *Pixbuf* en image sous format *PNG*. Cependant, nous avons eu des difficultés avec le chemin de l'image, car en fonction du dossier dans lequel la fonction est exécutée, l'image est introuvable. Il a donc fallu écrire une fonction qui construit le chemin absolu de l'image, nommée `get_image_path(char *filename)` pour que l'image puisse être trouvée depuis n'importe où dans le projet.

Rotation

La rotation était, elle aussi, complexe, car non seulement il fallait faire pivoter l'image d'un certain angle pour avoir le pixel d'arrivée, mais il fallait également le faire correspondre avec le pixel de départ, en faisant une rotation inverse. De plus, il fallait trouver la bonne formule avec le cosinus et le sinus de l'angle pour trouver le pixel correspondant à la rotation.

Découpage des images

La principale difficulté rencontrée au début de ce projet fut la gestion de l'image, à travers les différentes couches de *GTK* et *GDK* et les différents pointeurs. À cela s'ajoutait la complexité de programmer en *C* avec la gestion manuelle de la mémoire, l'allocation nécessaire pour stocker tous les *PixBuf* des différentes images découpées. L'idée de départ pour les nouvelles images ajustées était de les enregistrer sous un format *png*, mais nous nous sommes rendu compte que cela était bien trop coûteux en mémoire, car cela impliquait d'enregistrer une première fois l'image en *png*, puis la remettre en *PixBuf* pour enfin la remettre à nouveau en *png*, tout en ayant en tête que chercher et charger un fichier est assez coûteux. La solution a donc été d'allouer un tableau de pointeurs de *PixBuf* qui seront ensuite réutilisés dans d'autres fonctions.

2.2 Détection des lignes

Au début de la chaîne de traitement, nous avons des photos de différentes grilles de mots cachés et nous devons les "numériser" en détectant les caractères de la grille grâce à un futur *OCR*. Mais l'*OCR* sera entraîné pour analyser des lettres et non des fichiers entiers, donc il est nécessaire d'extraire chacune des lettres, ainsi que les coordonnées de la grille, de la liste de mots ainsi que chaque mot de la liste de mots.

2.2.1 Bilan des réalisations

Pour ce faire, j'ai utilisé différentes fonctions créées par les autres membres du projet ou par moi-même. Voici la chaîne de traitement du programme permettant de détecter les lettres.

Premièrement, je crée deux *Pixbufs*, tous les deux issus de l'image en entrée. Le premier me permettra de trouver les coordonnées de chaque lettre en effectuant plusieurs transformations dessus, et je couperai le deuxième aux coordonnées retournées par le premier. Les transformations effectuées sur le premier sont les suivantes : je convertis l'image en nuances de gris et ensuite je binarise l'image. Ensuite, je parcours horizontalement et verticalement le tableau pour retirer, s'il y en a, les lignes de la grille.

Pour détecter les lettres, je parcours l'image pixel par pixel, jusqu'à trouver un pixel noir. Quand j'en trouve un, je parcours tous ses pixels noirs voisins jusqu'à ne plus trouver de pixels noirs. Pour garder en mémoire les pixels que j'ai visités ou non, j'ai créé un tableau de la même taille que le nombre de pixels de l'image, qui garde un entier à la valeur soit zéro, soit un, et qui agit comme un booléen.

Avant de faire le parcours, j'ai fait une fonction pour détecter si une image est tournée, mais qui pour l'instant renvoie un angle de zéro degré. Cela me permettra de plus facilement implémenter cette fonction dans le futur.

Maintenant que j'ai les coordonnées de chaque lettre, je peux trouver les coordonnées de la grille ainsi que de la liste de mots. Pour ce faire, je parcours chaque lettre de la liste de lettres trouvées, et si une lettre se trouve dans l'intervalle $[x_{\min} - \text{seuil}; x_{\max} + \text{seuil}]$ et $[y_{\min} - \text{seuil}; y_{\max} + \text{seuil}]$ d'un des deux ensembles, alors j'en déduis qu'elle appartient à cet ensemble. (Je pars du principe que la liste de mots est à une distance raisonnable de la grille de lettres).

Enfin, comme j'obtiens les lettres appartenant à la liste de mots, je peux reparcourir ces lettres et établir des mini-ensembles représentant chaque mot de la liste, et pour savoir quelle lettre appartient à quel mot, je procède de la même manière que précédemment, mais uniquement sur l'axe x.

P	X	U	T	S	I	N	I	U	P	R	V	G	B	M	D	D
E	H	A	A	S	P	O	J	P	E	T	B	E	Q	Z	L	C
A	U	N	T	E	G	Q	T	L	H	R	Z	F	A	T	O	P
S	H	X	F	N	G	U	A	X	E	A	A	Y	P	O	M	H
Y	O	Y	Y	L	D	X	L	A	K	Y	U	Z	L	B	S	K
J	X	M	U	U	G	Q	T	R	I	M	A	G	I	N	E	B
H	F	N	W	F	X	H	D	P	B	B	B	T	N	V	S	K
H	I	I	H	D	E	S	Q	F	U	M	Y	E	R	N	S	X
R	P	B	Z	N	H	S	D	S	L	H	O	N	B	S	S	S
E	H	X	A	I	Z	I	H	A	H	O	E	S	Q	F	E	F
C	W	Z	I	M	V	D	C	J	V	S	S	I	M	G	R	W
L	A	I	I	R	Z	Q	Q	H	X	D	Z	O	Z	Q	T	R
W	C	A	X	E	Z	R	G	H	A	I	Z	N	E	C	S	E
B	R	H	F	O	T	G	N	I	T	S	E	R	E	O	V	Z
M	W	V	W	Q	D	U	I	H	W	Q	T	S	B	I	M	L
T	D	T	O	N	Z	C	X	X	R	G	E	L	K	H	F	Q
Q	N	E	K	S	V	M	O	T	F	A	L	A	A	E	W	B

(a) La grille découpée

IMAGINE
RELAX
COOL
RESTING
BREATHE
EASY
TENSION
STRESS
CALM

(b) La liste de mots découpée

Figure 5: Quelques images découpées: La grille et la liste de mots

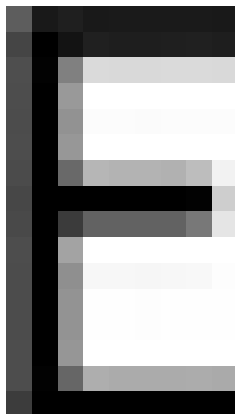
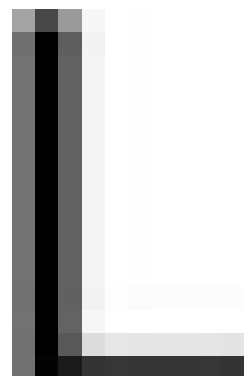
(a) La lettre E au pixel $x = 338, y = 100$ (b) La lettre L au pixel $x = 338, y = 169$

Figure 6: Quelques lettres découpées dans la grille

2.2.2 Défis rencontrés

Normalement, j'appliquais aussi un *filtre de Sobel* sur l'image pour détecter les contours de chaque élément, cela me permettait de mieux les découper par la suite, mais après plusieurs tests, je me suis rendu compte que ce n'était pas utile et que sans ce filtre la découpe se faisait assez bien. Donc le développement d'une fonction pour appliquer un *filtre de Sobel* sur une image m'a pris du temps, mais cela m'a permis de me familiariser avec les *Pixbufs*. Un autre problème qui apparaît est que les images du niveau deux et trois contiennent des dessins et/ou sont tournées sur un côté, cela entraîne un dysfonctionnement dans mon algorithme pour trouver les mots ainsi que la grille, car dans le cas d'une rotation de l'image, toutes les lettres d'une même ligne n'ont pas les mêmes coordonnées en ordonnée, et pour les images contenant des dessins, ceux-ci sont détectés comme des lettres et donc rendent les coordonnées de la grille et/ou de la liste de mots invalide. C'est pourquoi il faudrait appliquer certains filtres sur l'image de façon plus radicale pour ne garder que les lettres et mieux épurer l'image des groupements de pixels ne faisant pas référence à des lettres.

2.3 Réseau de neurones

2.3.1 Bilan des réalisations

Dans le cadre de ce projet, nous avons implémenté un *POC* de notre réseau de neurones artificiels en langage C, capable d'apprendre différentes fonctions logiques binaires, telles que *XOR*, *AND*, *OR* et *XNOR*. L'objectif était de mettre en pratique les concepts fondamentaux des réseaux de neurones, notamment le *perceptron*, la *fonction d'activation*, la *rétropropagation* et l'optimisation des poids par *descente de gradient*.

Pour chaque fonction logique, le réseau comporte deux neurones en entrée correspondant aux variables logiques, une couche cachée de deux neurones utilisant la fonction *sigmoïde* afin d'introduire la non-linéarité nécessaire pour apprendre le *XOR* et le *NXOR*, et une couche de sortie composée d'un unique neurone également sigmoïdal. La fonction sigmoïde a été choisie pour sa continuité et sa dérivabilité, indispensables à la *rétropropagation*, et pour sa capacité à produire des valeurs entre 0 et 1 (fonction binaire comparée à *softmax*), correspondant naturellement aux valeurs logiques.

Les poids et les biais ont été initialisés aléatoirement afin de briser toute symétrie et permettre un apprentissage efficace. La *rétropropagation* a permis d'ajuster les poids en fonction de l'erreur entre les sorties prédites et les valeurs cibles. Le taux d'apprentissage a été fixé expérimentalement à 0,5, ce qui a permis une convergence rapide tout en évitant les oscillations. Les tests ont montré que le réseau pouvait correctement reproduire les tables de vérité des fonctions *AND*, *OR* et *XNOR*, et qu'il réussissait également à apprendre le *XOR*, démontrant la capacité du réseau à modéliser des relations non linéaires.

Fonction	Expression	Entrée 00	Entrée 01	Entrée 10	Entrée 11
AND	$A \cdot B$	0	0	0	1
OR	$A + B$	0	1	1	1
XOR	$A \oplus B$	0	1	1	0
XNOR	$\overline{A} \cdot \overline{B} + A \cdot B$	1	0	0	1

Table 1: Tables de vérité des fonctions logiques utilisées pour l'apprentissage du réseau de neurones.

```
--- Tests XNOR (!A.!B + A.B) ---  
Epoch 0 - Error: 1.049113  
Epoch 1000 - Error: 0.393433  
Epoch 2000 - Error: 0.013107  
Epoch 3000 - Error: 0.006015  
Epoch 4000 - Error: 0.003846  
Epoch 5000 - Error: 0.002810  
Epoch 6000 - Error: 0.002208  
Epoch 7000 - Error: 0.001815  
Epoch 8000 - Error: 0.001539  
Epoch 9000 - Error: 0.001334  
Input: 0 0, Predicted: 1, Expected: 1  
Input: 0 1, Predicted: 0, Expected: 0  
Input: 1 0, Predicted: 0, Expected: 0  
Input: 1 1, Predicted: 1, Expected: 1  
All tests passed.
```

Figure 7: Exemple de prédiction du réseau de neurones

2.3.2 Défis rencontrés

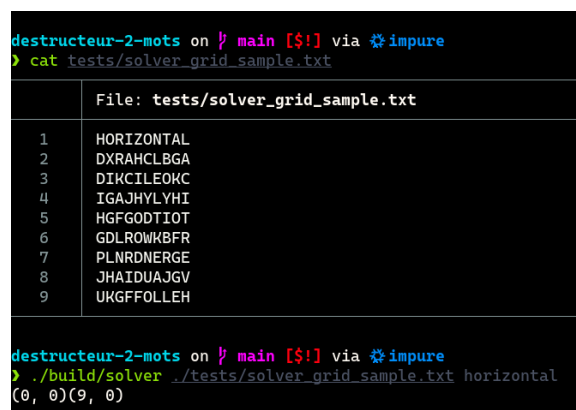
L'une des principales difficultés de ce projet réside dans la compréhension et la gestion de multiples abstractions mathématiques et informatiques. La conception d'un réseau de neurones implique de manipuler des notions telles que les vecteurs de poids, les fonctions d'activation non linéaires, la rétropropagation et les dérivées partielles pour calculer les gradients. Cette complexité se cumule avec la nécessité de programmer en *C*, où la gestion manuelle de la mémoire et la structuration correcte des matrices et vecteurs de poids sont critiques. Comprendre comment chaque neurone contribue à la sortie finale et comment l'erreur se propage à travers les couches demande une forte capacité d'abstraction et une vision claire du flux des données. L'ajustement des hyperparamètres, tels que le taux d'apprentissage et l'initialisation des poids, ajoute un niveau de complexité supplémentaire, car ils influencent directement la convergence et la stabilité de l'apprentissage. Ces aspects ont nécessité plusieurs itérations expérimentales et un suivi attentif pour s'assurer que le réseau converge correctement vers la solution attendue.

2.4 Algorithme de résolution

Ce projet a pour but final de résoudre une grille de mots cachés en trouvant les coordonnées de chacun des mots dedans. C'est pourquoi il est obligatoire d'avoir, à la fin de la chaîne de traitement du projet, un algorithme prenant en entrée un tableau de caractères ainsi qu'une liste de mots, et d'avoir en sortie les coordonnées de chacun des mots dans la grille.

2.4.1 Bilan des réalisations

Ici, comme nous voulons tester notre *solver* avec des commandes, nous devons prendre en entrée un mot ainsi qu'un fichier texte qui contient le tableau de caractères. Donc dans la fonction *solver*, nous devons transformer le fichier texte en un tableau de caractères pour ensuite effectuer la recherche dans ce tableau. Je commence par effectuer un parcours ligne par ligne jusqu'à tomber sur la même lettre que la première lettre du mot à chercher, et ensuite, à partir de ce point-là, un parcours dans toutes les directions est lancé, s'arrêtant s'il trouve le mot en entier ou si aucune direction n'a donné de résultat satisfaisant. Si le mot n'est pas trouvé, alors l'algorithme continue son parcours jusqu'à trouver le mot ou à arriver à la fin du tableau, ce qui signifie que la recherche est négative.



```
destructeur-2-mots on 1 main [!] via impure
> cat tests/solver_grid_sample.txt
File: tests/solver_grid_sample.txt
1 HORIZONTAL
2 DXRAHCLBGA
3 DIKCILEOKC
4 IGAJHYLYHI
5 HGFGODTIOT
6 GDLROWKBFR
7 PLNRDNERGE
8 JHAIDUJGV
9 UKGFFOLLEH
destructeur-2-mots on 1 main [!] via impure
> ./build/solver ./tests/solver_grid_sample.txt horizontal
(0, 0)(9, 0)
```

Figure 8: Démonstration du fonctionnement de l'algorithme de résolution

2.4.2 Défis rencontrés

Lors de la réalisation de cet algorithme, plus précisément lors du passage d'un fichier texte à un tableau de caractères, une imprécision a ralenti le développement. En effet, Windows ou Linux n'enregistre pas les fins de ligne de la même façon, Windows met un `\n\r` à la fin et Linux met seulement un `\n`. Cela rendait l'algorithme non fonctionnel pour certains membres du groupe, mais pas pour tout le monde. Mais cette erreur mineure a été rapidement trouvée et réglée. Pour ce qui est du parcours dans toutes les directions, le code est très long, car il se répète huit fois pour les huit directions différentes, c'est pourquoi il est envisageable de modifier l'architecture de cette partie pour la rendre plus concise.

2.5 Interface graphique

L'interface graphique permet au projet de prendre forme en permettant de voir le résultat de nos fonctions. Elle permet donc de tester le bon fonctionnement du projet en visualisant le résultat de la transformation d'une image par une fonction de traitement d'image, comme les conversions en niveaux de gris, ou en noir et blanc. Nous avons donc décidé d'avancer sur cette interface graphique pour la première soutenance, ce qui nous a permis de vérifier que le traitement d'image est fonctionnel.

2.5.1 Bilan des réalisations

L'interface graphique se lance avec une fenêtre *GtkWindow*, connectée à une application *GtkApplication*. Elle permet de charger une image, et d'y appliquer diverses opérations de traitement d'image:

- Conversion en niveaux de gris
- Conversion en noir et blanc
- Rotation de l'image

En plus de ces trois opérations de traitement d'image, l'interface graphique permet également de:

- Réinitialiser l'image (efface toutes les transformations appliquées à l'image)
- Sauvegarder l'image vers un fichier PNG
- Fermer la fenêtre

Elle se lance avec `./build/ui <optionnel: chemin de l'image>`, et charge l'image donnée en paramètre, ou l'image `level_1_image_1.png` par défaut. L'utilisateur peut donc cliquer sur les boutons pour effectuer les transformations et voir les changements directement sur la fenêtre.

Enfin, les boutons appellent les fonctions respectives (par exemple, la conversion en niveau de gris appelle `convert_to_grayscale()`) et appellent ensuite les fonctions `update_image()` et `apply_transformations()` pour mettre à jour l'image affichée dans la fenêtre. Les boutons indiquent à l'utilisateur le traitement effectué, comme par exemple la sauvegarde de l'image avec le nom du fichier de l'image sauvegardée, pour pouvoir y accéder par la suite.

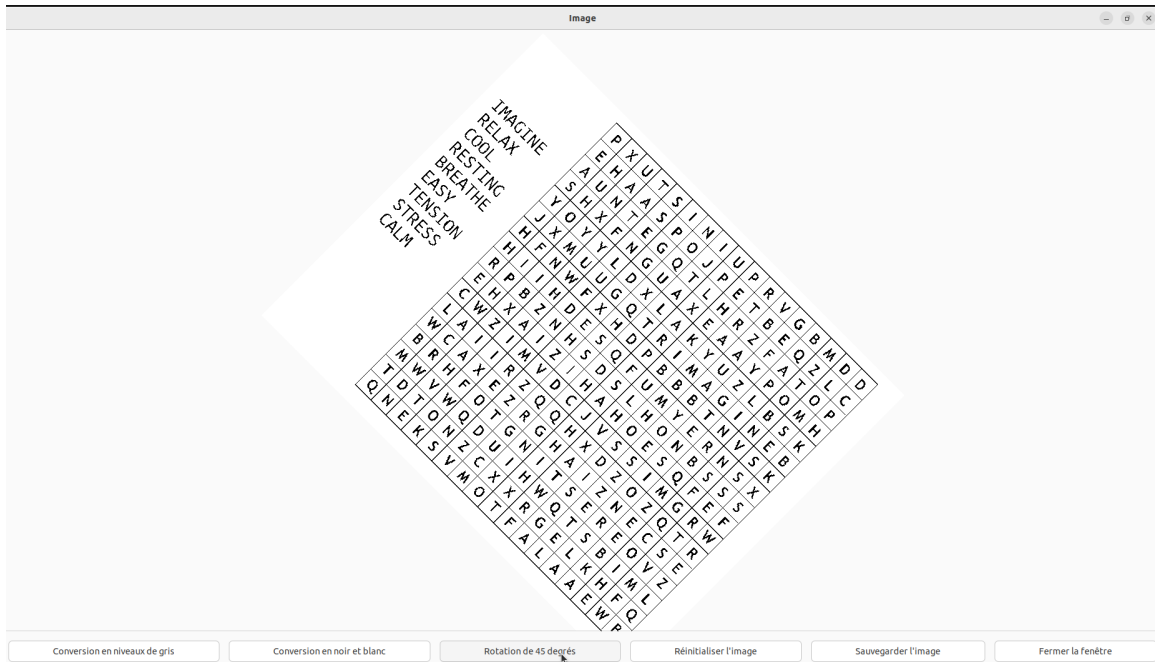


Figure 9: Interface graphique avec chargement de l'image *level_1_image_1.png*, conversion en noir et blanc puis rotation de 45 degrés vers la droite

2.5.2 Défis rencontrés

La complexité du développement de l'interface graphique réside en la gestion des pointeurs pour chaque bouton, mais aussi la gestion des pointeurs pour les conteneurs. En effet, il vaut veiller à ce que chaque bouton soit connecté à la bonne fonction et qu'il soit empilé dans le bon ordre dans les boîtes. De plus, comme l'utilisateur doit pouvoir effectuer plusieurs traitements à la fois sur une même image, nous avons créé une structure *AppData* qui contient des pointeurs sur les *Pixbufs* des images originelle et transformée.

3 Gestion de projet

3.1 Répartition et avancement du travail

Domaines de travail	Répartition				Avancement	
	Kristen	Romain	Liam	Timothée	Novembre	Décembre
Soutenances	R				30%	100%
Réseau de neurones	R				30%	100%
Structure et conception	R				0%	100%
Reconnaissance des lettres	R				0%	100%
Sauvegarde et chargement de l'entraînement	R				0%	100%
Traitement de l'image			R	S	90%	100%
Chargement et affichage d'une image			R	S	100%	100%
Suppression des couleurs			R	S	100%	100%
Rotation			R	S	80%	100%
Détection de ligne		R			90%	100%
Détection de la position grille		R			90%	100%
Détection de la position liste de mots		R			90%	100%
Détection des cases et lettres		R			90%	100%
Résolution de la grille		S		R	50%	100%
Reconstruction de la grille				R	0%	100%
Reconstruction de la liste de mots				R	0%	100%
Résolution de la grille		S		R	80%	100%
Interface graphique	S		R		10%	100%

Table 2: R correspond au rôle de Responsable et S correspond au rôle de Suppléant. Les sous-catégories détaillent les étapes du projet.

4 Conclusion

Nous avons posé les fondations techniques du projet en développant un prototype fonctionnel de réseau de neurones capable d'apprendre des fonctions logiques de base, en concevant un premier algorithme de résolution de grilles et en mettant en place les outils essentiels au traitement et à la manipulation d'images.

Cette première phase nous a permis d'acquérir une compréhension approfondie des concepts clés liés au projet, notamment la *rétropropagation*, la manipulation d'images avec *GTK/GDK*, et la gestion de la mémoire en C. Les défis rencontrés, qu'ils soient mathématiques, algorithmiques ou techniques, ont renforcé notre rigueur et notre coordination au sein de l'équipe.

Pour la suite du projet, l'objectif sera de connecter l'ensemble de ces modules et de les compléter afin de créer un programme fonctionnel et fluide : de la lecture d'une image brute jusqu'à l'affichage d'une grille entièrement résolue.

En somme, cette première soutenance valide notre compréhension technique du projet. Les bases posées aujourd'hui serviront à construire, d'ici à la soutenance finale, une application complète, performante et capable de résoudre automatiquement des grilles de mots cachés à partir d'images.

5 Annexes

5.0.1 Termes techniques et définitions

PoC : *Proof of Concept*, désigne une version préliminaire d'un projet permettant de valider la faisabilité technique d'une idée avant son développement complet.

OCR : *Optical Character Recognition*, technologie permettant de reconnaître et de convertir du texte présent dans une image en texte éditable.

GTK : *GIMP Toolkit*, bibliothèque utilisée pour créer des interfaces graphiques (GUI) en C, compatible avec Linux, Windows et macOS.

GDK : *GIMP Drawing Kit*, couche intermédiaire entre GTK et le système de fenêtrage, utilisée pour la gestion des événements et le rendu graphique de bas niveau.

XOR/AND/OR/XNOR : Opérateurs logiques utilisés dans les circuits et réseaux de neurones pour combiner des valeurs binaires selon des règles booléennes.

Rétropropagation : Algorithme d'apprentissage supervisé qui ajuste les poids d'un réseau de neurones en fonction de l'erreur calculée entre la sortie attendue et la sortie réelle.

Descente de gradient : Méthode d'optimisation consistant à modifier progressivement les poids d'un modèle afin de minimiser une fonction de coût.

Perceptron : Neurone artificiel simple qui calcule une somme pondérée de ses entrées, applique une fonction d'activation et produit une sortie binaire.

Fonction d'activation : Fonction mathématique appliquée à la sortie d'un neurone pour introduire de la non-linéarité et permettre l'apprentissage de relations complexes.

GdkPixbuf : Structure de données de la bibliothèque GDK utilisée pour représenter et manipuler des images en mémoire (chargement, affichage, sauvegarde).

PNG : *Portable Network Graphics*, format d'image sans perte qui prend en charge la transparence et est couramment utilisé pour les interfaces et le web.

Filtre de Sobel : Opération de détection de contours qui calcule le gradient d'intensité d'une image afin de mettre en évidence les zones de transition rapide de luminosité.

5.0.2 Bibliographie

- *Neural Networks And Deep Learning* : <http://neuralnetworksanddeeplearning.com/>
- *Sausheong's article* : <https://sausheong.github.io/posts/how-to-build-a-simple-artifi>
- *Mots mêlés* : https://en.wikipedia.org/wiki/Word_search
- *Détection de ligne* : https://en.wikipedia.org/wiki/Line_detection
- *Documentation GTK3* : <https://docs.gtk.org/gtk3/>
- *nicolas.thiery.name* : <https://nicolas.thiery.name/Enseignement/Info111/Projet-Image/partie-3.html>